STM32定时器与PWM实验报告

芯片：STM32F103C8T6，开发环境：STM32CubeMX + CLion，使用PA1引脚（TIM2_CH2），LED直接连接PA1与GND（无串联电阻）

一、实验目的

1. 理解STM32通用定时器（TIM2）的工作原理，掌握定时器中断的配置流程。
2. 学会使用定时器中断实现GPIO翻转，控制LED周期性亮灭。
3. 掌握PWM脉宽调制原理，学会配置定时器PWM输出通道，实现LED呼吸渐变效果。
4. 熟悉STM32CubeMX图形化配置、CLion编译、STM32CubeProgrammer烧录的完整开发流程。

二、实验原理

任务1：定时器中断实现LED周期闪烁

TIM2是STM32F103的通用定时器，可以对内部时钟计数，当计数值到达自动重载值ARR时，产生更新溢出事件。开启定时器更新中断后，触发中断回调函数，在回调函数内翻转PA1引脚电平。

本实验设置闪烁周期2s，LED亮1s、灭1s。

公式：定时器溢出频率 f{*TIM}= *frac{f*{APB1}}{(PSC+1)(ARR+1)}

系统时钟72MHz，APB1定时器时钟72MHz。配置PSC=7199，ARR=999，定时器溢出频率10Hz（100ms中断一次），计数10次翻转IO，实现1s定时。

任务2：TIM2 PWM输出呼吸灯

PWM（脉冲宽度调制）：周期固定，高电平占比（占空比）可调。PA1复用为TIM2_CH2通道，定时器工作在PWM生成模式。不断修改比较寄存器CCR的值，改变PWM占空比。占空比越大LED越亮，占空比越小LED越暗，循环增减CCR数值，实现渐亮渐灭的呼吸效果，完整周期约2s。

硬件说明：LED阳极接PA1，阴极接GND，无限流电阻，直接驱动。

三、实验硬件与软件环境

硬件：STM32F103C8T6最小系统板、LED灯、ST-LINK下载器、杜邦线、USB供电线

软件：STM32CubeMX、CLion、STM32CubeProgrammer

芯片：STM32F103C8T6，使用TIM2、PA1引脚(TIM2_CH2)

四、实验步骤

4.1 任务一：定时器中断，2s周期LED闪烁

1. CubeMX工程新建，选择STM32F103C8T6芯片。
2. 引脚配置：PA1设置为GPIO_Output推挽输出。
3. 配置TIM2：选择内部时钟源Internal Clock，PSC=7199，ARR=999；NVIC勾选TIM2全局中断。
4. 时钟树配置：HSE外部晶振8MHz，系统时钟SYSCLK=72MHz。
5. 生成代码，CLion打开工程。
6. 在main.c用户代码区添加全局变量与定时器中断回调函数。
7. 在main函数初始化后，启动定时器中断：HAL_TIM_Base_Start_IT(&htim2);
8. 编译生成.hex文件，使用STM32CubeProgrammer烧录。
9. 单独USB上电，观察LED：亮1s，灭1s，周期2s循环闪烁。

   任务一核心代码片段

```c
void HAL_TIM_Base_MspInit(TIM_HandleTypeDef* tim_baseHandle)
{

  if(tim_baseHandle->Instance==TIM2)
  {
  /* USER CODE BEGIN TIM2_MspInit 0 */

  /* USER CODE END TIM2_MspInit 0 */
    /* TIM2 clock enable */
    __HAL_RCC_TIM2_CLK_ENABLE();

    /* TIM2 interrupt Init */
    HAL_NVIC_SetPriority(TIM2_IRQn, PreemptPriority: 0, SubPriority: 1);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
  /* USER CODE BEGIN TIM2_MspInit 1 */

  /* USER CODE END TIM2_MspInit 1 */
  }
}

void HAL_TIM_Base_MspDeInit(TIM_HandleTypeDef* tim_baseHandle)
{

  if(tim_baseHandle->Instance==TIM2)
  {
  /* USER CODE BEGIN TIM2_MspDeInit 0 */

  /* USER CODE END TIM2_MspDeInit 0 */
    /* Peripheral clock disable */
    __HAL_RCC_TIM2_CLK_DISABLE();

    /* TIM2 interrupt Deinit */
    HAL_NVIC_DisableIRQ(TIM2_IRQn);
  /* USER CODE BEGIN TIM2_MspDeInit 1 */

  /* USER CODE END TIM2_MspDeInit 1 */
  }
}
```

4.2 任务二：TIM2 PWM呼吸灯（周期约2s）

10. 重新打开ioc配置文件，PA1引脚修改复用功能TIM2_CH2，模式Alternate function push-pull复用推挽输出。
11. TIM2配置为PWM Generation CH2，PSC=71，ARR=999，PWM频率1kHz；NVIC开启TIM2更新中断。
12. 生成工程代码，CLion打开main.c，删除旧GPIO翻转代码。
13. 添加PWM全局变量，在main函数中同时启动PWM通道与定时器更新中断。
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
    HAL_TIM_Base_Start_IT(&htim2);
14. 在定时器中断回调函数修改CCR比较值，动态调整占空比，实现呼吸渐变。
15. 编译，生成hex，CubeProgrammer烧录。
16. 单独上电，LED从暗逐渐变亮，再从亮慢慢变暗，循环往复，完整周期约2s。

17. Keil虚拟示波器：可以接入调试，观测PWM波形随占空比变化的波形。

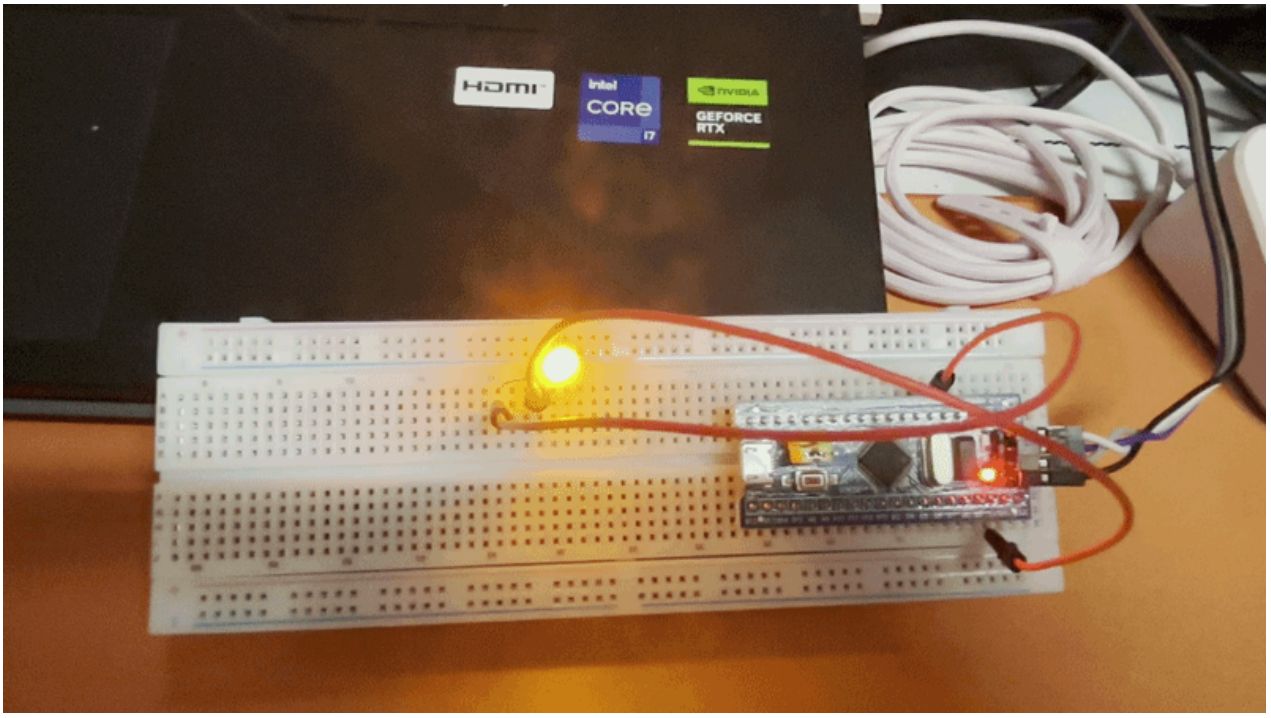
任务二核心代码片段

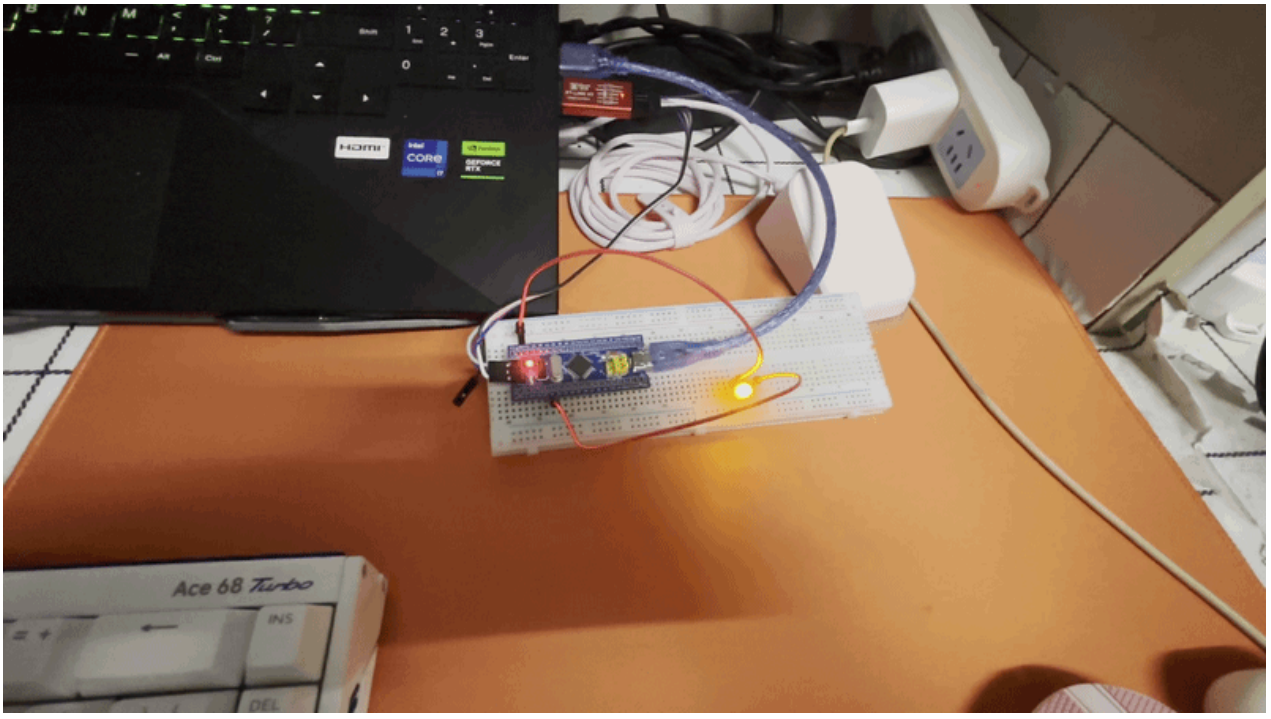
```c
/* USER CODE BEGIN 0 */
uint16_t pwm_val = 0;
int8_t dir = 1;

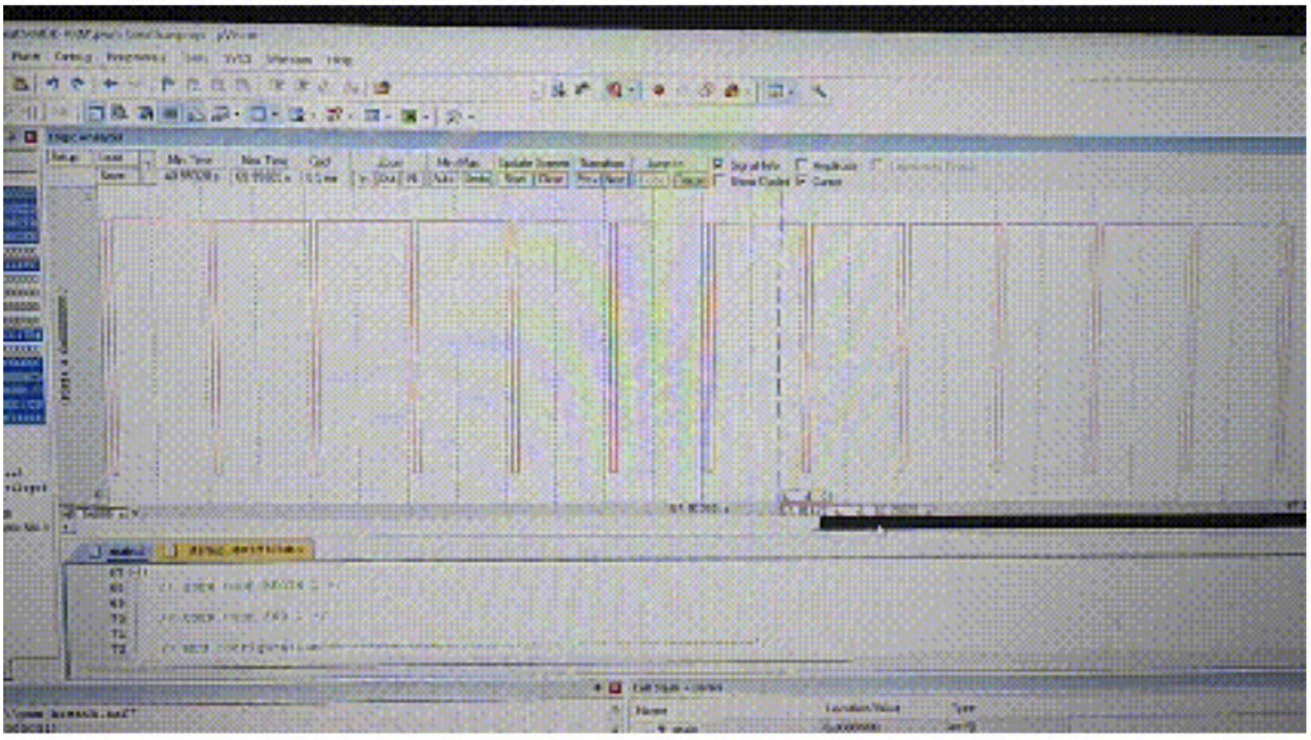
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if(htim->Instance == TIM2)
  {
    pwm_val += dir;
    if(pwm_val >= 999)
    {
      dir = -1;
    }
    else if(pwm_val <= 0)
    {
      dir = 1;
    }
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pwm_val);
  }
}
/* USER CODE END 0 */
```

五、实验现象

18. 任务1现象：上电后LED以2s为周期周期性亮灭，亮1秒，熄灭1秒，稳定循环。按下复位按键，重新开始计时闪烁。

19. 

20. 任务2现象：上电后LED亮度平滑渐变，从暗慢慢变到最亮，随后平滑变暗直到熄灭，循环往复，一个完整呼吸周期约2s。肉眼看不到频闪，亮度连续变化。
21. 



六、实验结果分析

1. 定时器中断依靠硬件计数器自动计数，不需要CPU持续循环延时，属于非阻塞定时，CPU在等待计数时可以执行其他任务，相比HAL_Delay阻塞延时效率更高。
2. PWM不是真正模拟电压，是快速高低电平切换。人眼视觉暂留效应，平均等效电压随占空比变化，表现为LED亮度变化。
3. 硬件注意：本实验LED直接连接PA1，没有串联限流电阻，属于简易测试方式，长期使用存在烧毁IO口风险。
4. 调试问题记录：烧录出现擦除Flash失败，是因为芯片运行旧程序；通过BOOT0进入ISP模式，解除写保护后完成烧录。

七、实验总结

本次实验完成了STM32通用定时器两个典型应用：定时器中断定时翻转IO实现周期闪烁；定时器PWM输出实现LED呼吸灯。掌握了CubeMX图形配置定时器、中断、PWM通道，学会HAL库定时器相关API，理解PWM的原理。熟悉CLion编译、CubeProgrammer烧录的完整开发流程，掌握嵌入式调试的基本排错思路。